University of BRISTOL

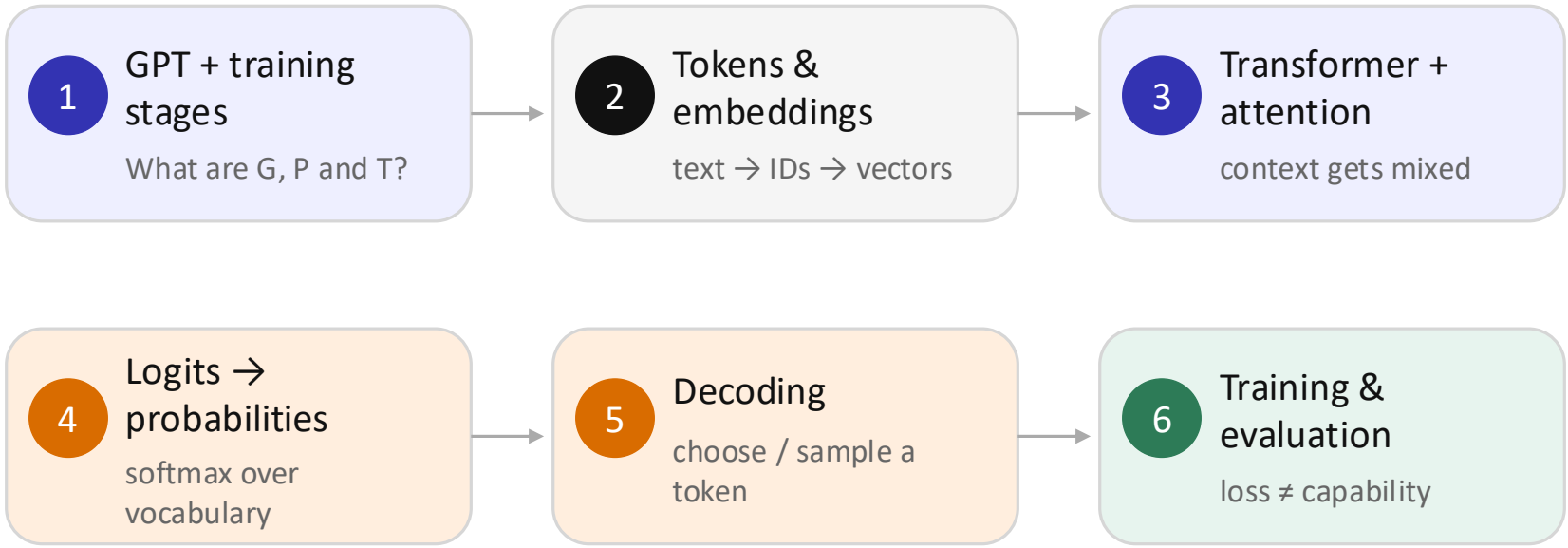
COMS20018, Introduction to AI:
Large Language Models:
Pre-training, Tokens, Attention & Decoding

Nan Lu

School of Computer Science
University of Bristol

12th October 2026

Agenda



Today's question

How does GPT predict the next token, and how do we train and evaluate that predictor?

A question to begin with

Do you know what GPT in ChatGPT stand for?

G Generative

Predict a distribution for the next token, then repeat. (recap L06 autoregressive generation)

Bristol is a → city?

P Pre-trained

Learn general language patterns from massive text before your task.

text / code → base model

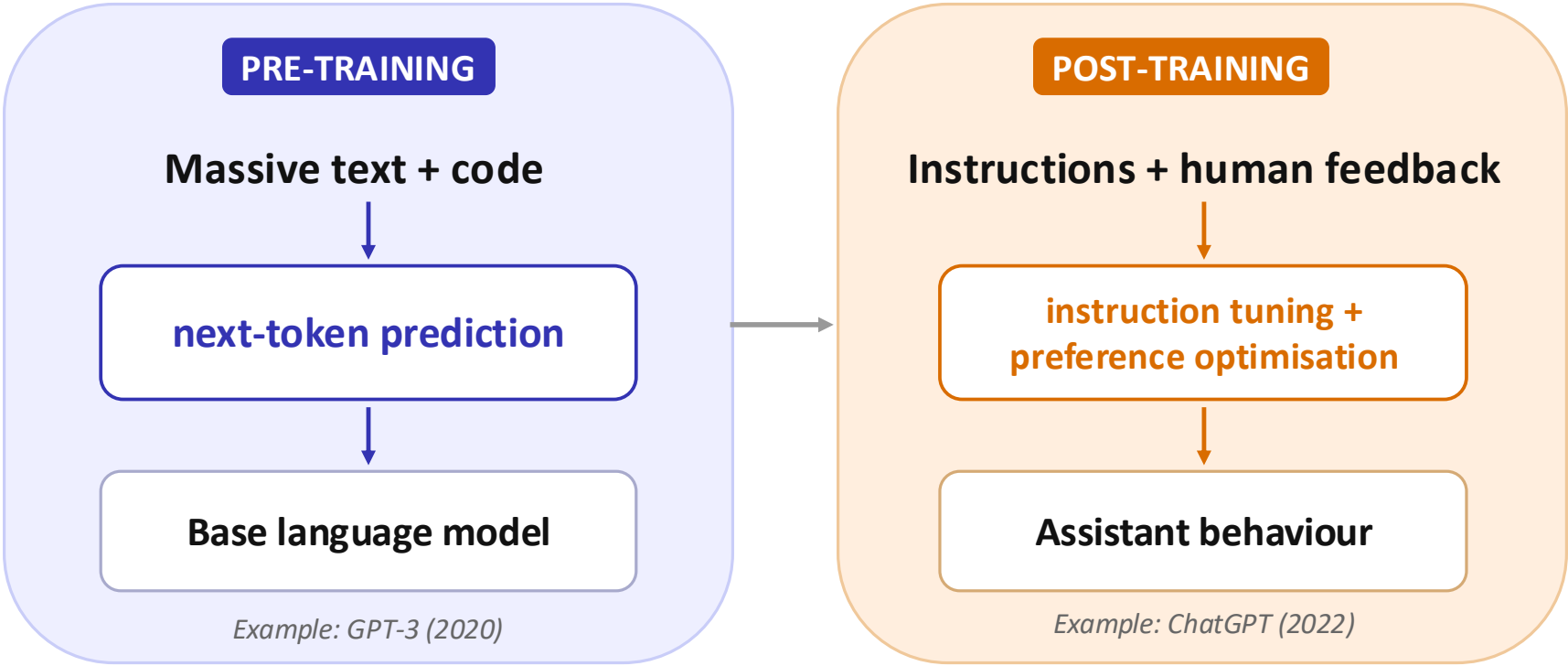
T Transformer

Use attention to combine the parts of the context that matter (recap L05 attention)

Bristol … because it …

use relevant earlier context

Pretraining vs. Post-training



Pre-training learns to model text.

Post-training learns how we want the model to respond to users.

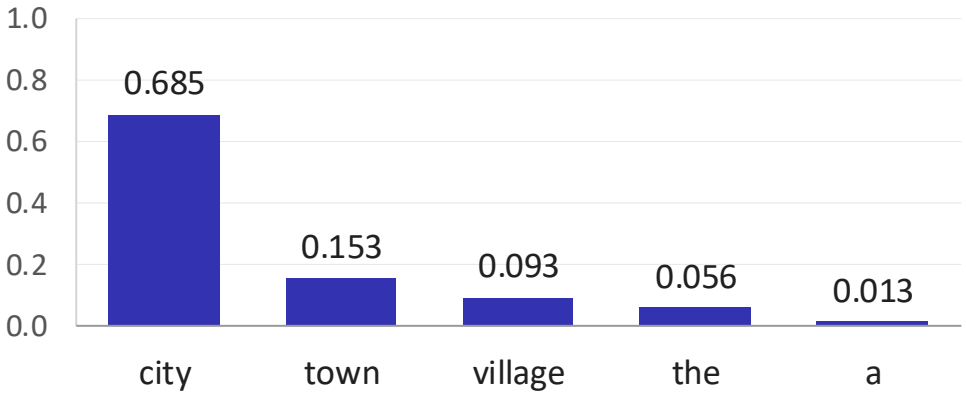
Pretraining objective: predict the next token

Training text: “Bristol is a city in England.”

Context
“Bristol is a”

Observed next token
“ city”

Predicted p(next token | “Bristol is a”)



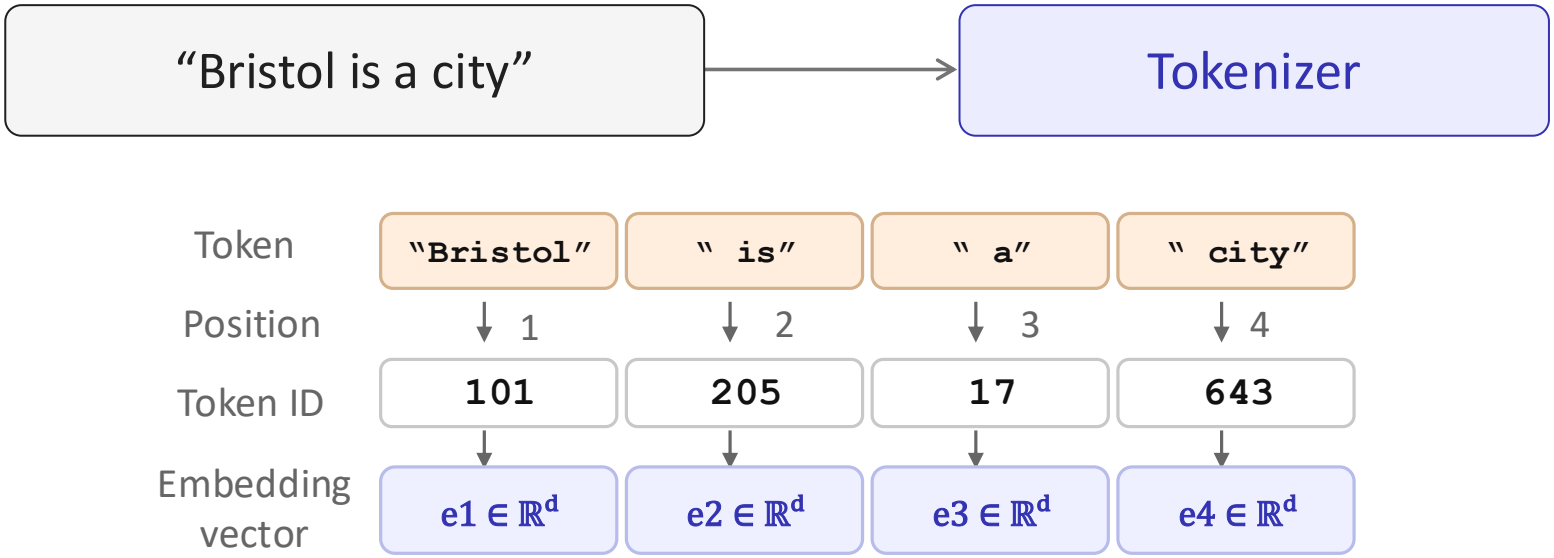
- **Training**: learn to predict the next token in training text by adjusting model parameters.
- **Generation**: choose a next token, append it to context, and repeat with parameters fixed.

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_{<t})$$

Toy examples throughout: word-like tokens, invented IDs, and a five-token output vocabulary.

Text → tokens → numbers

A neural network cannot consume characters. Text is first cut into *tokens*.



- **Text → tokens**: a tokenizer splits text into pieces from a fixed vocabulary; a token may be a word, part of a word, or punctuation.
- **Tokens → vectors**: each token has an integer ID; the ID selects a learned vector from the embedding table.
- **Word order**: position information is added to help the model distinguish "dog bites man" ≠ "man bites dog".

Same token embedding, different contexts

Bristol is a city

Same " city", ID 643

Token embedding:

$e(city) = (0.2, 0.6, -0.1, 0.4)$

"Bristol is a city"

Contextual representation (hidden state):

$h_4 = (0.8, 0.1, 0.6, -0.2)$

Paris is a city

Same " city", ID 643

Token embedding:

$e(city) = (0.2, 0.6, -0.1, 0.4)$

"Paris is a city"

Contextual representation (hidden state):

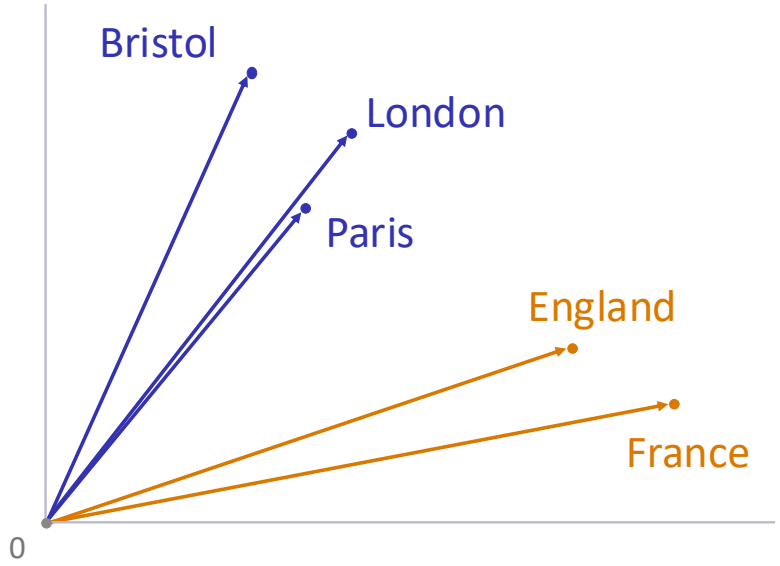
$h_4 = (-0.1, 0.9, 0.2, 0.7)$

The embedding identifies the token. The Transformer (will introduce soon) builds a representation that also reflects its context.

Learned embeddings: structure and meaning

1 Structure

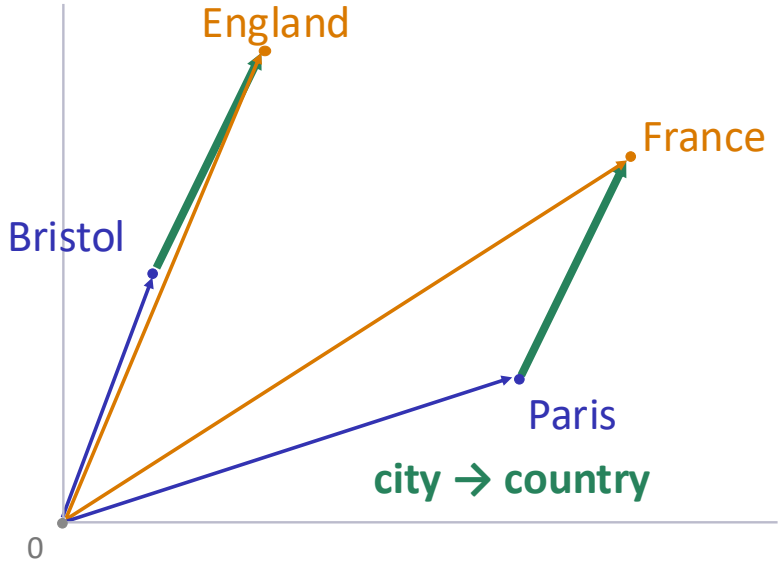
Similar tokens can have nearby vectors.



Neighbourhoods reveal useful similarities.

2 Semantic information

Directions can encode a shared relation.

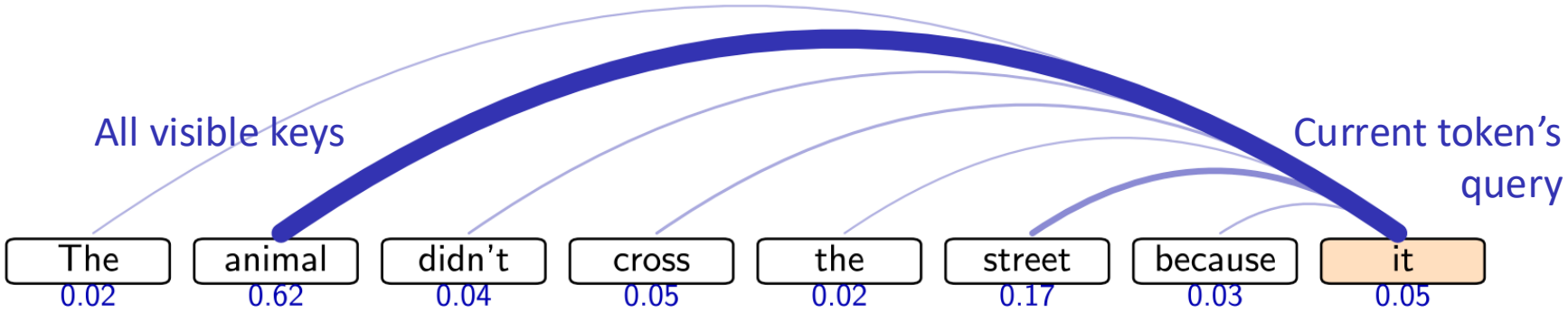


Related pairs can share a similar offset.

Why it helps: the model can reuse learned patterns across related words.

Attention: gathering information from context

At “it”, which visible tokens provide useful information?



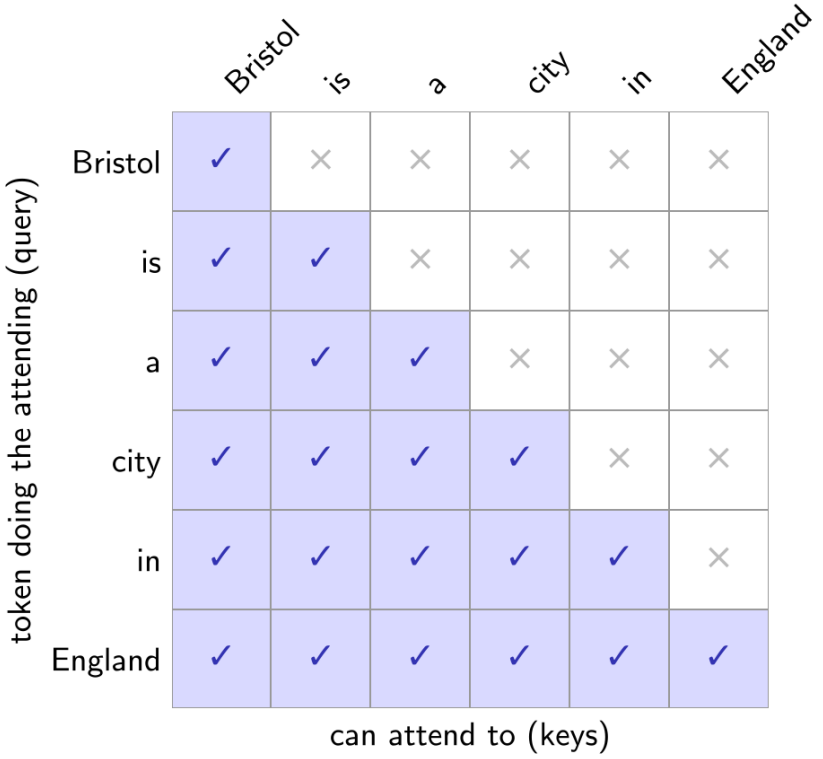
- Each token’s current representation is projected into three vectors:
  - **Query** (what information am I looking for): $q_i = x_i W_Q$
  - **Key** (what can this token be matched on): $k_j = x_j W_K$
  - **Value** (what information can this token contribute): $v_j = x_j W_V$
- Match the query to keys, then combine values:

$$\alpha_{ij} = \text{softmax}_j\left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}\right) \qquad \boldsymbol{\alpha}_i = \sum_j \alpha_{ij}\, \mathbf{v}_j$$

Notations: i: current position (“it”); j: a visible position; x: input to this attention layer; W_Q, W_K, W_V :learned matrices; d_k :query/key dimension

Causal attention: each position can only look backwards

Keys: context positions



- Each position sees itself and earlier tokens. Future positions receive zero attention weight.
- At “a”, the model uses “Bristol is a” to predict “city”.
- During training, the mask prevents the model from seeing the next-token answer.
- One forward pass per token: cheaper, but not free.
- All positions can be trained in parallel while keeping future tokens hidden.

Try it: an attention output

For the final position in “Bristol is a”:

Visible token	**Attention weight** α_{ij}	**Value vector** $\mathbf{v}_j$
Bristol	0.5	(2, 0)
is	0.3	(0, 2)
a	0.2	(4, 4)

Try it (3 min)

1. Check that the weights form a valid attention distribution.
2. Calculate the three weighted value vectors $\alpha_{ij}\ \mathbf{v}_j$.
3. Calculate the attention output $\alpha_{\mathrm{i}} = \alpha_{\mathrm{i}1}\mathbf{v}_1 + \alpha_{\mathrm{i}2}\mathbf{v}_2 + \alpha_{\mathrm{i}3}\mathbf{v}_3$.

Try it: an attention output (answer)

1. All weights are non-negative and $0.5 + 0.3 + 0.2 = 1$: a valid distribution.

2. Weighted value vectors:

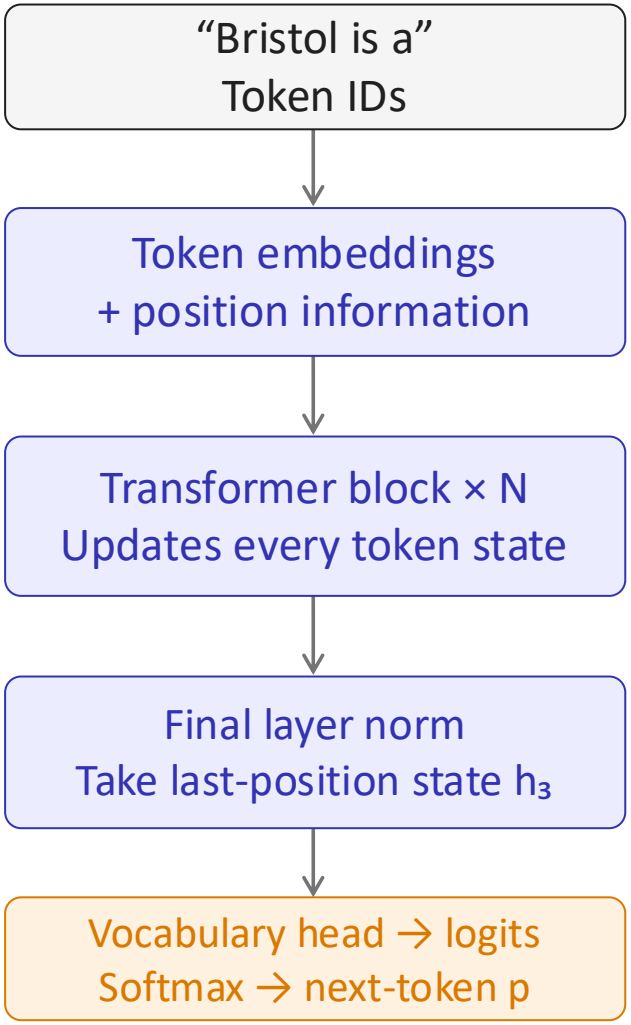
$$0.5\begin{bmatrix}2\\0\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix}, \qquad 0.3\begin{bmatrix}0\\2\end{bmatrix} = \begin{bmatrix}0\\0.6\end{bmatrix}, \qquad 0.2\begin{bmatrix}4\\4\end{bmatrix} = \begin{bmatrix}0.8\\0.8\end{bmatrix}$$

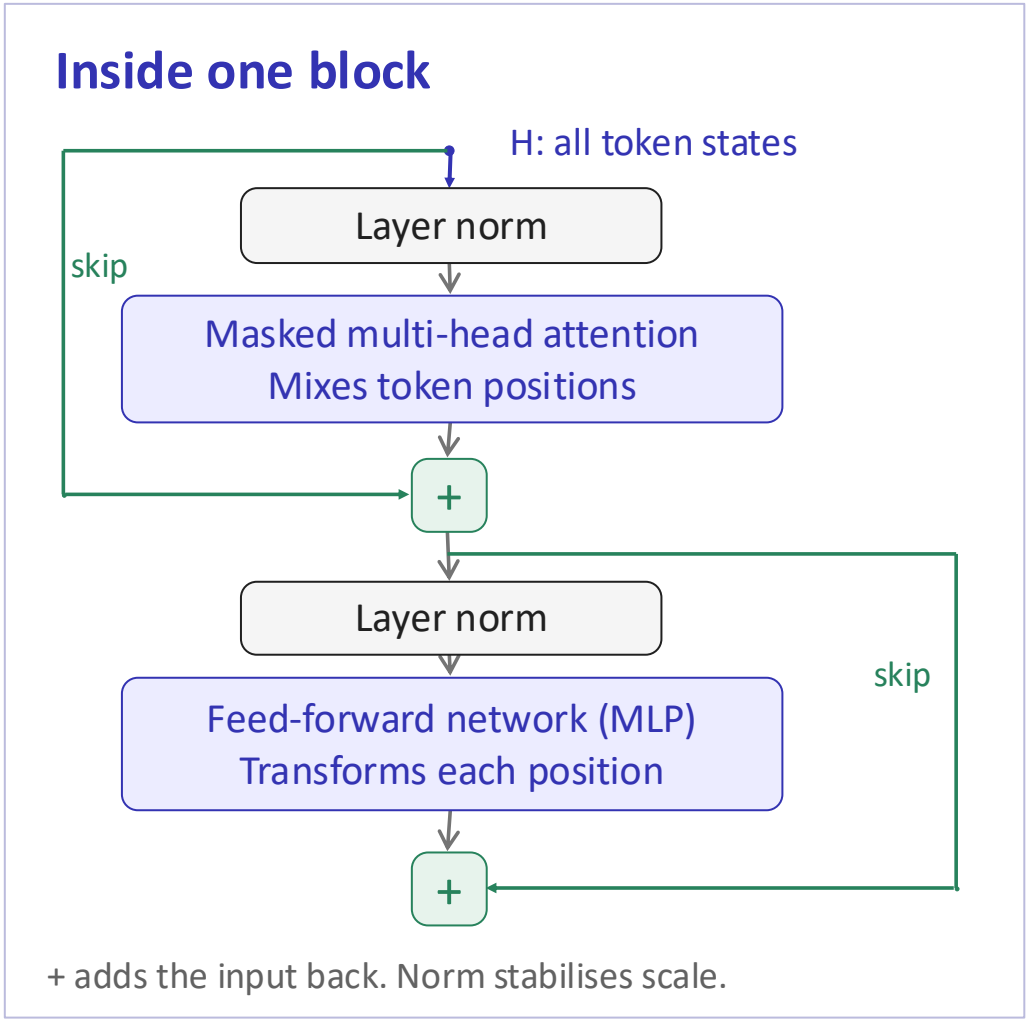
3. Add them:

$$\begin{bmatrix}1 + 0 + 0.8\\0 + 0.6 + 0.8\end{bmatrix} = \begin{bmatrix}1.8\\1.4\end{bmatrix}$$

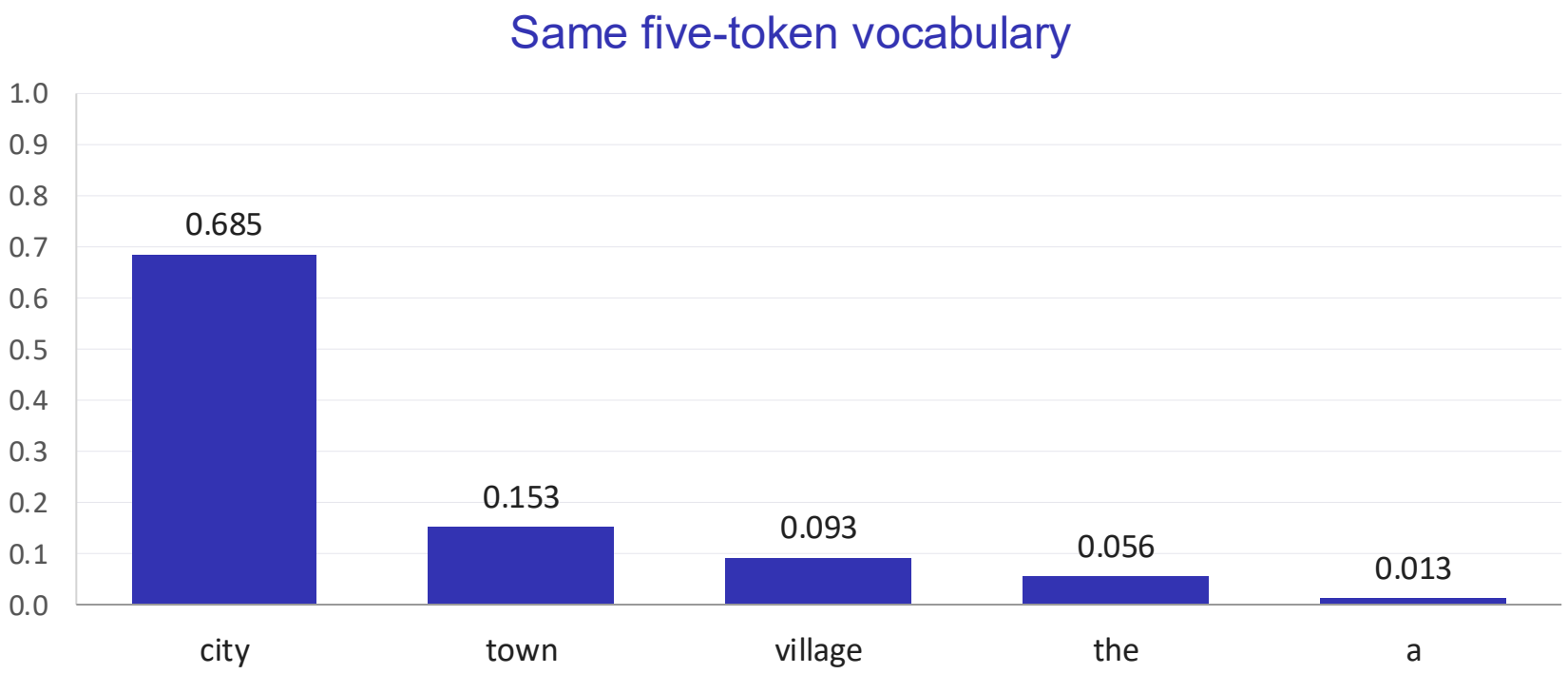
Transformer structure: a stack of decoder blocks

GPT-style decoder-only model (pre-norm example)





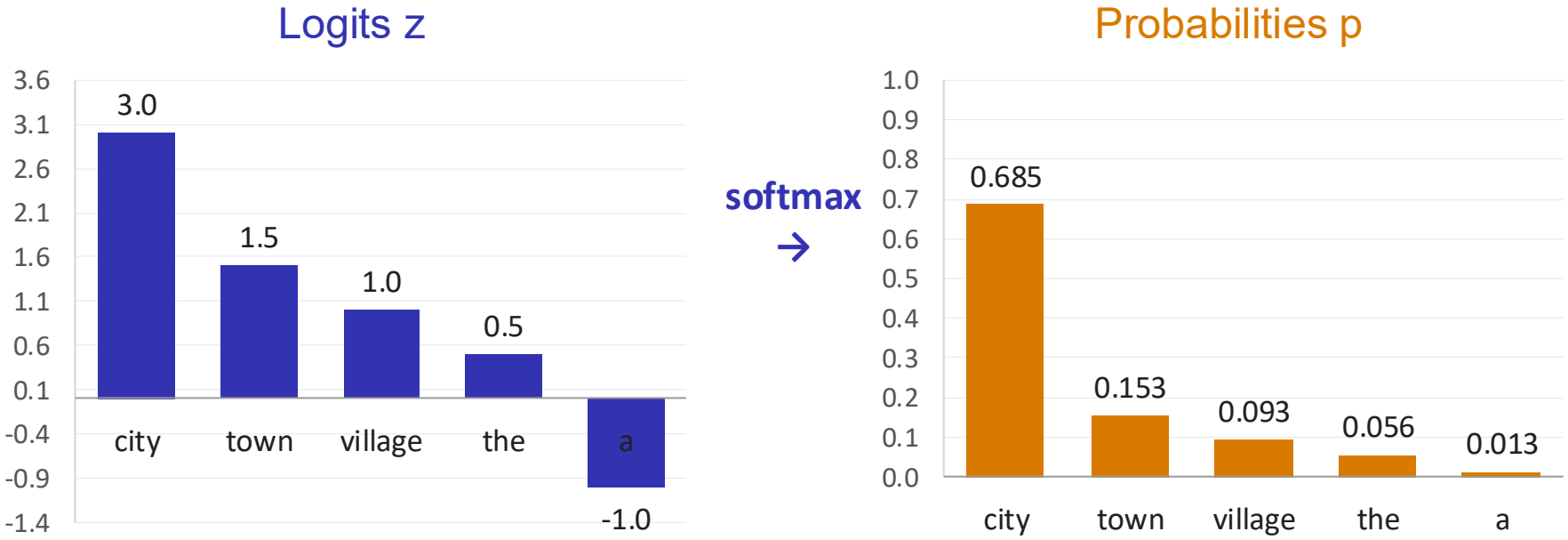
The next-token distribution for “Bristol is a”



- “city” has the highest probability. Other tokens remain possible.
- Softmax turns scores into probabilities.
- Decoding selects a token from that distribution.

From logits to probabilities: softmax

Context: “Bristol is a”; same tokens on both sides



$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i}}{Z}$$

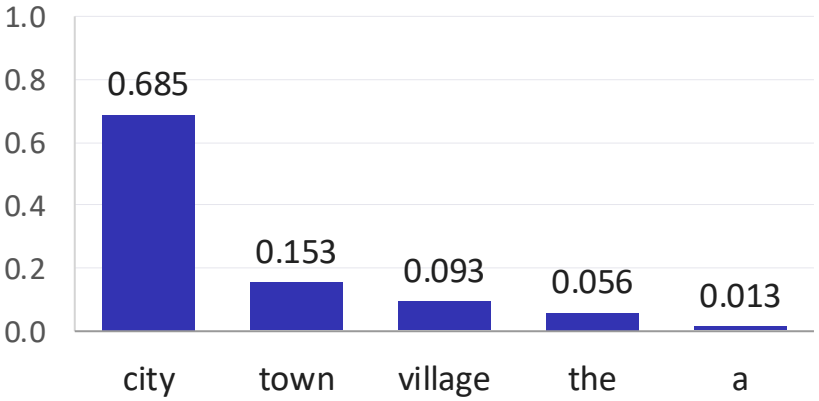
Exponentiate → sum → normalise

- Negative logits are allowed. Probabilities are positive and sum to 1.
- The ranking stays the same; the scale changes.

Choosing the next token: greedy or sample?

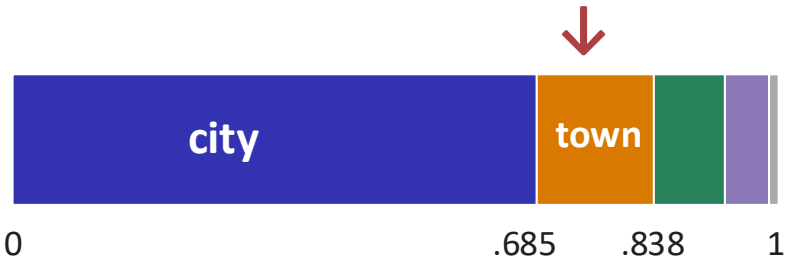
Context: “Bristol is a”

Greedy decoding: choose the largest



$$argmax_i\, p_i = city$$

Sampling: draw on [0, 1)



Village / the / a occupy the small segments.

May select “ town”

- Greedy always selects “city” for these fixed probabilities.
- Sampling can select another token. Neither rule guarantees truth.

Try it: softmax by hand

Prompt: “Bristol is a”

Token	Logit z
city	3.0
town	1.5
village	1.0
the	0.5
a	-1.0

Try it (5 min)

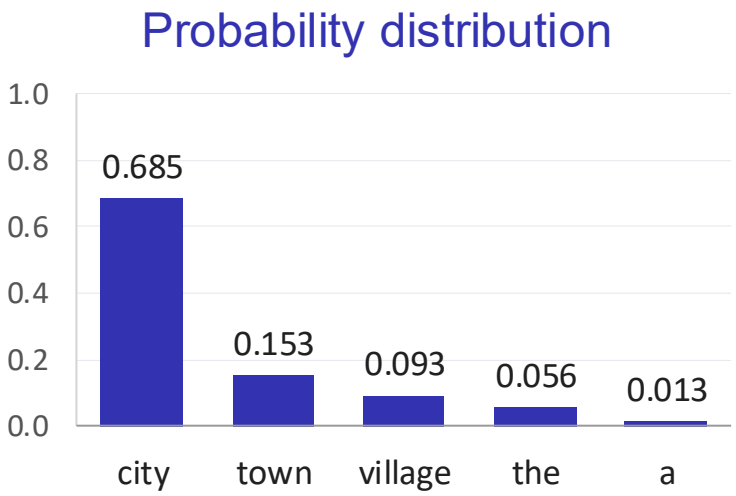
► Round exponentials, Z and probabilities to three decimal places.

1. Calculate e^{z_i} for all five tokens.
2. Calculate the normalising constant Z.
3. Calculate the full next-token probability distribution.
4. Which token does greedy decoding (argmax) select?

Try it: softmax by hand (answer)

1. Exponentiate each logit

Token	Logit	exp(z)	p
city	3.0	20.086	0.685
town	1.5	4.482	0.153
village	1.0	2.718	0.093
the	0.5	1.649	0.056
a	−1.0	0.368	0.013



2. Add the exponentials to get the normaliser

$$Z \approx 20.086 + 4.482 + 2.718 + 1.649 + 0.368 = 29.303$$

3. Divide each exponential by Z

$$p(city) \approx \frac{20.086}{29.303} \approx 0.685 \qquad p(town) \approx \frac{4.482}{29.303} \approx 0.153$$

Apply the same division to every token. Greedy selects “city”.

Temperature: same ranking, different spread

Divide every logit by a temperature $T > 0$ before the softmax:

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T = 1$: the standard softmax
- $T < 1$: differences grow; the distribution gets ***sharper***
- $T > 1$: differences shrink; the distribution gets ***flatter***

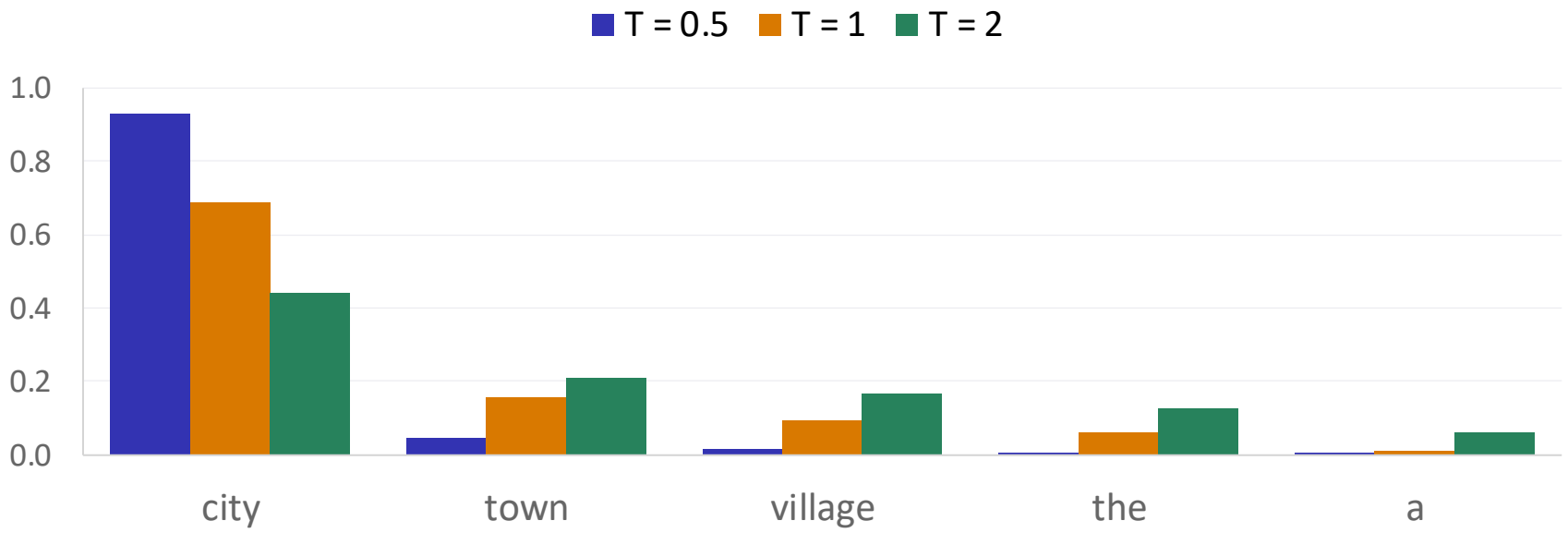
Try it (3 min)

- Starting again from the Bristol logits $(3.0, 1.5, 1.0, 0.5, -1.0)$:

1. Calculate the scaled logits z_i/T and the full distribution for $T = 2$ (three decimal places).

2. Compare $P(\text{“ town”})$ with your answer for $T = 1$, and explain the change.

Temperature: same ranking, different spread (answer)



At T = 2: scaled logits = (1.5, .75, .5, .25, −.5)

Z = 10.138, and p(“ town”) rises from .153 to .209.

“city” remains the top token.

Lower temperature means less randomness, not greater truth.

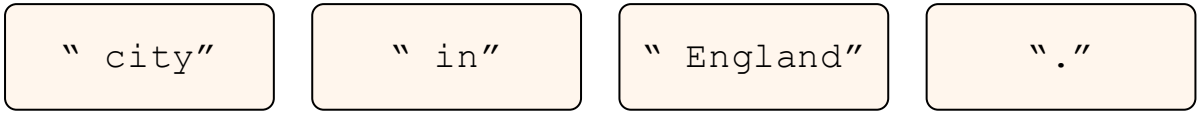
Generating one token at a time

Step	Context available	Generated token
1	Bristol	is
2	Bristol is	a
3	Bristol is a	city
4	Bristol is a city	in
5	Bristol is a city in	England
6	Bristol is a city in England	.

- Six new tokens require six token-prediction steps.
- Each selected token becomes context for the next step.

Try it: counting forward passes

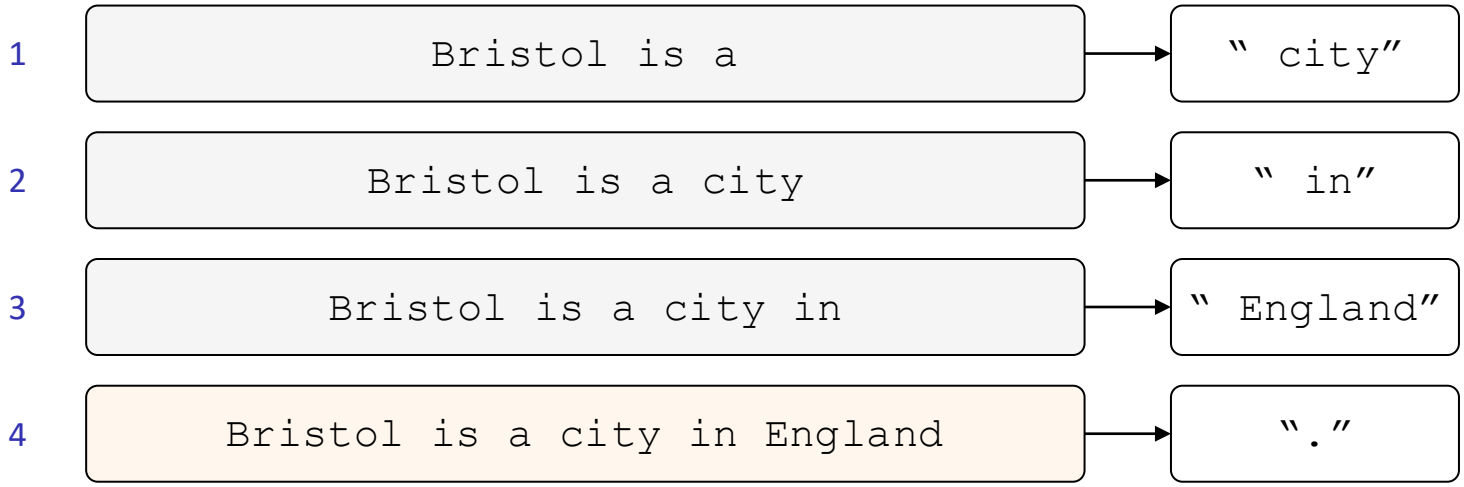
Prompt: `Bristol is a`. Each new token needs one forward pass, and tokens are generated one after another. The model generates four new tokens:



Try it (2 min)

1. How many forward passes are needed?
2. What is the input on the *first* forward pass?
3. What is the input on the *final* forward pass?

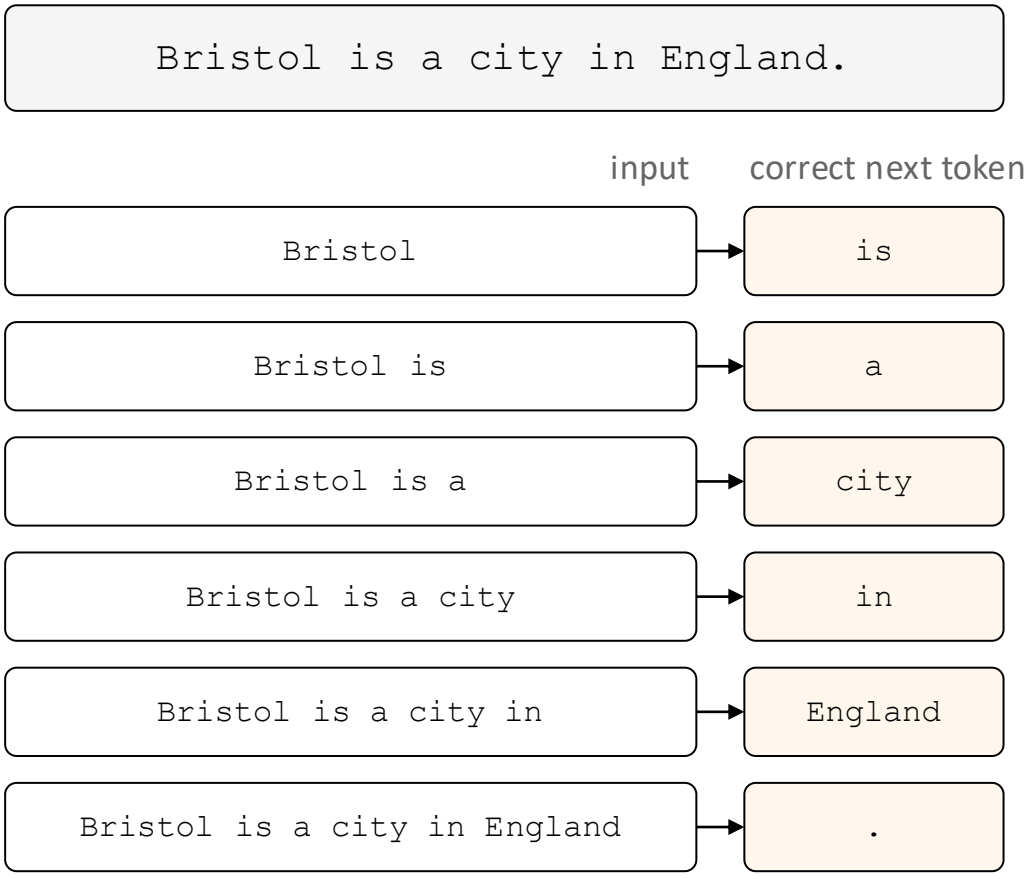
Try it: counting forward passes (answer)



1. **Four** forward passes: one per generated token.
2. First pass: just the prompt, `Bristol is a`.
3. Final pass: the prompt followed by the first *three* generated tokens, `Bristol is a city in England`. It predicts the fourth token, “.”.

⇒ In general: generating n tokens takes n passes, and the last input is the prompt plus the first $n-1$ generated tokens.

Training: one sentence, many examples



- The text supplies its own next-token targets: self-supervised learning.
- With a causal mask, one training forward pass scores all six positions.
- Backpropagation then updates the parameters.

⇒ Data quality matters because the targets come from the text.

The training signal: cross-entropy

Context: “Bristol is a” Observed next token: “city”

$$L(y,p) = -\sum_i y_i \ln p_i = -\ln p(city \mid Bristol\ is\ a)$$

p_i : predicted probability. y_i: 1 for “city”, 0 for all other tokens.

Token	**city**	**town**	**village**	**the**	**a**
Target y	**1**	0	0	0	0
Prediction p	**0.685**	**0.153**	**0.093**	**0.056**	**0.013**
Prediction p’	**0.093**	**0.153**	**0.685**	**0.056**	**0.013**

Try it (2 min)

- Calculate the cross-entropy loss for prediction p and prediction p’.

The training signal: cross-entropy (answer)

Same context. Same target: “city”. Different predictions.

Prediction p:

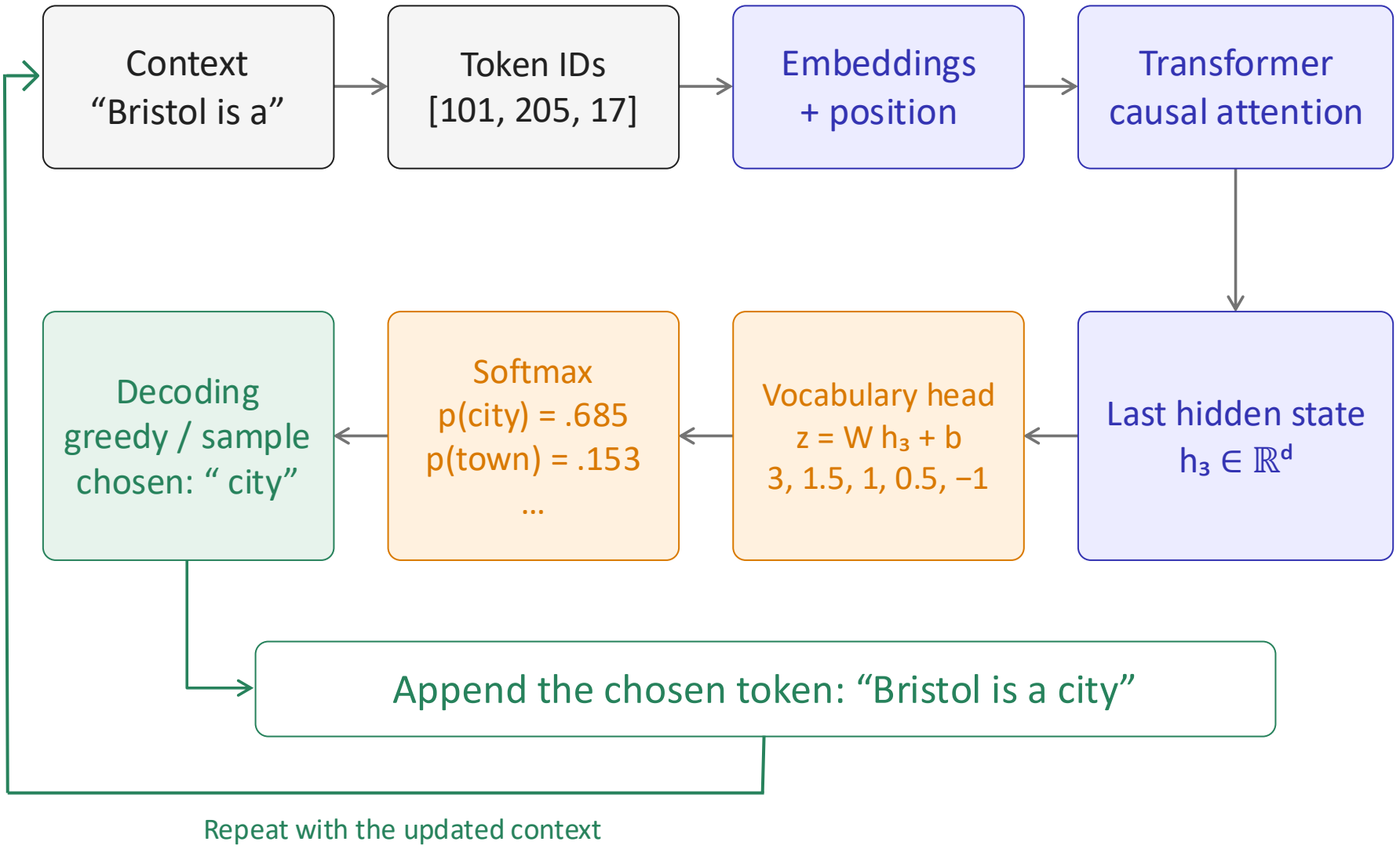
$$p(city) = 0.685 \Rightarrow \mathrm{L(y, p)} = -\ln(0.685) \approx 0.378$$

Prediction p′:

$$p'(city) = 0.093 \Rightarrow L(y, p') = -ln(0.093) \approx 2.375$$

Cross-entropy loss penalises assigning low probability to the target next token.

Big picture: how a GPT model generates the next token



Training vs. inference

Training

- Learns from a training dataset
- Pretraining and possible post-training
- Parameters change through optimisation
- Produces updated weights

Inference (generation)

- Uses the available context
- Parameters stay fixed
- Predicts and selects tokens
- Can reuse cached computation

When you “teach” a chatbot something in a conversation,
you are not changing the model. You are changing its **context**.

⇒ This distinction runs through the whole of L08.

Summary

Five things to remember

1. A GPT model predicts a distribution over the next token.
2. Learned embeddings represent tokens as useful vectors.
3. Causal attention builds representations from prior context.
4. Softmax gives probabilities; decoding selects a token.
5. Training changes parameters; inference uses fixed weights.

You should now be able to calculate

An attention output, softmax probabilities, temperature effects, cross-entropy loss, and autoregressive generation steps.

Reading

Core reading: Transformers and Pretraining

Jurafsky & Martin, Speech and Language Processing, Ch. 7.
Focus: decoder blocks, next-token training and decoding.

Tokens and embeddings

Same free textbook: Ch. 2 (Words and Tokens) and Ch. 5 (Embeddings). Read these to revisit the input representations.

Visual companion: The Illustrated GPT-2

Jay Alammar. Follow a decoder-only model from token embeddings through masked attention to next-token scores.

Optional practice: softmax and cross-entropy

Dive into Deep Learning, §4.1 (Softmax Regression).
Connect probabilities, one-hot targets and negative log loss.